CS 342 — Software Design

Project 2: Three Card Poker — JavaFX Application Design

Team Size: Individual or teams of two students

Part 1 Due: Sunday, November 1, 2026, at 11:59 PM: no late submissions
Part 2 Due: Sunday, November 15, 2026, at 11:59 PM: 24 hours late for 10% penalty

Submission: Canvas

---

1. Project Overview

In this project, you will design and implement a graphical version of Three Card Poker using JavaFX.

The primary focus of this project is **software design, UI/UX design, event-driven programming, and automated testing**.

You will develop a multi-screen JavaFX application using FXML for interface layouts, CSS for visual styling, and Java controllers to handle user interactions.

Your application must support two players playing independently against a dealer.

The project consists of two parts:

- **Part 1: Interface Design and Usability Evaluation.** Create wireframes, plan your application architecture, test your wireframes with other students, and revise your designs based on their feedback.
- **Part 2: Implementation and Testing.** Implement the game using JavaFX, incorporate animations and multiple screens, and write automated tests for the game logic.

The goal is to build an application that is not only functionally correct but also intuitive, visually consistent, and maintainable.

2. Learning Objectives

By completing this project, you should be able to:

1. Apply software design principles, including separation of concerns and encapsulation.
2. Design a graphical interface using wireframes and iterative user feedback.
3. Use FXML and JavaFX controllers to implement a multi-screen application.
4. Apply UI/UX principles such as consistency, feedback, error prevention, accessibility, and visual hierarchy.
5. Implement event-driven application behavior using JavaFX event handlers.
6. Use JavaFX animations to improve the user experience.
7. Apply CSS to create consistent and adaptable visual styles.
8. Separate game rules and business logic from presentation code.
9. Test application logic using JUnit Jupiter.
10. Use Maven to manage dependencies, build the application, and run tests.

Part 1 — Interface Design and Usability Evaluation

Due: Sunday, November 1, 2026, at 11:59 PM

Required Screens

Before implementing the application, design its user interface.

Your application must contain **at least three distinct screens**. Each screen should support a different stage of the user's interaction with the game.

Screen 1: Welcome / Main Menu

The welcome screen introduces the application and allows users to begin playing.

It should include:

- Game title and visual identity.
- A clear Start Game control.
- Access to game rules or instructions.
- An appropriate way to exit the application.

Screen 2: Game Play

The game screen provides the primary interaction with the application.

It should include:

- Both players' cards.
- Dealer cards, initially hidden.
- Ante and Pair Plus betting controls.
- Play and Fold controls.
- Current wagers and cumulative winnings.
- Game-phase information.
- Feedback explaining what the user should do next.

The design should make it clear which actions are available during each stage of the game.

Screen 3: Results / Round Summary

This screen presents the outcome of the completed round.

It should include:

- Dealer's final hand.
- Each player's final hand.
- Hand classifications.
- Winner information.
- Wager and payout results.
- Updated cumulative winnings.
- Controls to play another round or return to the main menu.

Additional screens, such as Settings or Help, are permitted.

The three required screens must be distinct interfaces rather than simply three pop-up dialogs over the same screen.

Wireframe Requirements

Create wireframes for each required screen.

Your wireframes must demonstrate:

- Placement of major UI elements.
- Navigation between screens.
- User input controls.
- Feedback and status messages.
- Enabled and disabled controls.
- Error prevention and recovery.
- Visual hierarchy.
- Intended animations or transitions.

Include at least one additional wireframe illustrating an alternate application state, such as an invalid wager, a folded hand, or a completed round.

You may use Figma, diagrams, presentation software, or another appropriate design tool including hand drawn.

Your wireframes should be detailed enough that another student can understand how to use the application without seeing the completed implementation.

Usability Testing with Other Students

You must conduct usability testing of your wireframes with **at least two students who are not members of your project team**.

The goal is to identify problems before implementation.

Testing Procedure

Ask each participant to complete realistic tasks using your wireframes.

Example tasks include:

1. Start a new game.
2. Locate the Ante and Pair Plus betting controls.
3. Determine how to continue playing or fold.
4. Find the results of a completed round.
5. Start another round or return to the main menu.

Avoid telling participants exactly where to click or what controls to use.

Observe how they interpret the interface.

Required Documentation

For each participant, record:

- Tasks attempted.
- Whether each task was completed successfully.
- Points of hesitation or confusion.
- Questions asked.
- Suggestions or feedback.

Across the sessions, identify at least **three usability findings**.

Based on the findings, make at least **two meaningful revisions** to your wireframes.

Document the original design, the revised design, and why each change was made.

Your revisions should address actual usability problems rather than merely changing colors or decorations.

Software Design Plan

Provide a brief written explanation of how you plan to organize your application.

Discuss the responsibilities of:

- Game model classes.
- Game logic and payout calculations.
- A game-service or coordination component.
- JavaFX FXML controllers.
- Screen navigation.

Explain how your architecture will keep game rules independent of the graphical interface.

A UML class diagram is **not required for Part 1**.

Part 1 Submission

Submit **one PDF file** to the Part 1 assignment in Canvas by **Sunday, November 1, 2026, at 11:59 PM**.

Your PDF must include:

1. Wireframes for all required screens.
2. At least one alternate-state wireframe.
3. A diagram showing navigation between screens.
4. A brief software architecture description.
5. Usability testing tasks, observations, and results.
6. At least three usability findings.
7. At least two documented wireframe revisions with before-and-after comparisons.
8. A short explanation of the UI/UX principles applied to your design.

All Part 1 materials must be combined into a single PDF.

You are not required to submit working JavaFX code for Part 1.

Part 2 — JavaFX Implementation and Testing

Due: Sunday, November 15, 2026, at 11:59 PM

Technology Requirements

Your completed application must use:

- Java 25.
- JavaFX 25.
- FXML for the three required screens.
- JavaFX controller classes.
- CSS for styling.
- Maven for building and running the project.
- JUnit Jupiter for automated testing.

The application must run using a documented Maven command, such as:

```
mvn javafx:run
```

The project must build and run without requiring IDE-specific settings.

Three Card Poker Game Rules

Your application must implement the following two-player version of Three Card Poker.

Each player competes independently against the dealer.

Betting

Each player must be able to:

- Place an Ante wager between $5 and $25.
- Optionally place a Pair Plus wager between $5 and $25.
- Place a Play wager equal to the Ante to continue after seeing their cards.
- Fold instead of placing the Play wager.

A player who folds forfeits their Ante and Pair Plus wagers.

Dealing

Each player and the dealer receive three cards.

Players can see their own cards before deciding whether to Play or Fold.

The dealer's cards remain hidden until the appropriate point in the round.

Dealer Qualification

The dealer must have at least Queen-high to qualify.

If the dealer does not qualify:

- The Play wager is returned.
- The Ante is carried forward according to the original project's rules.

If the dealer qualifies, the player's hand is compared against the dealer's hand to determine the outcome.

Hand Rankings

From highest to lowest:

1. Straight Flush
2. Three of a Kind
3. Straight
4. Flush
5. Pair
6. High Card

Your implementation must correctly evaluate hand rankings and resolve ties.

Pair Plus Payouts

Hand	Payout
Straight Flush	40:1
Three of a Kind	30:1
Straight	6:1
Flush	3:1
Pair	1:1

Pair Plus is evaluated independently of the dealer's hand for players who have not folded.

Software Architecture Requirements

Your implementation must separate game logic from interface behavior.

The following classes and responsibilities are required.

Card

Represents a playing card.

Each card must have a suit and value, with card values ranging from 2 through 14 (Ace = 14).

Deck

Represents a standard 52-card deck.

It must support deck initialization, shuffling, and resetting.

Dealer

Maintains the deck and deals three-card hands.

Player

Maintains information about a player, including:

- Current hand.
- Ante wager.
- Pair Plus wager.
- Play wager.
- Cumulative winnings.

ThreeCardLogic

Contains the game-rule and payout calculations.

The following public static methods are required:

```
public static int evalHand(ArrayList<Card> hand);

public static int evalPPWinnings(
    ArrayList<Card> hand, int bet);

public static int compareHands(
    ArrayList<Card> dealer,
    ArrayList<Card> player);
```

Preserve the return-value conventions from the original project.

Game logic must be testable without launching the JavaFX interface.

Game Coordination

Create a dedicated class or component responsible for managing the game state and round progression.

For example, a `GameService` may coordinate:

- Starting a game.
- Accepting wagers.
- Dealing cards.
- Recording Play and Fold decisions.
- Revealing the dealer's hand.
- Determining results.
- Updating winnings.
- Beginning the next round.

You may choose appropriate names and supporting classes.

JavaFX Controllers

Each required screen must have an FXML layout and an associated controller.

Controllers should handle user events, update interface elements, coordinate navigation, and invoke game operations.

Do not implement hand evaluation, payout calculations, or deck management directly inside controller classes.

User Interface and UX Requirements

Your implementation should follow the revised designs from Part 1.

Navigation

Users must be able to navigate between the required screens using clearly labeled controls.

The interface should prevent accidental loss of an active game.

Feedback

The application must clearly communicate:

- Current game phase.
- Available player actions.
- Invalid input.
- Player decisions.
- Dealer qualification.
- Round outcomes.
- Updated winnings.

Error Prevention

Prevent invalid actions whenever possible.

For example:

- Do not permit wagers outside the allowed range.
- Do not permit a Play action before the required wager has been placed.
- Disable actions that are not valid in the current game phase.
- Prevent starting another round before the current round is resolved.

Accessibility

Use readable text, sufficient color contrast, clear labels, visible keyboard focus, and keyboard-accessible controls.

Do not rely on color alone to communicate winning, losing, or invalid actions.

12. Animation Requirements

Your application must implement **at least three distinct, purposeful JavaFX animations**.

Animations should improve the user's understanding of game events rather than serve as decoration alone.

Possible examples include:

- Cards moving from the deck to player positions.
- Dealer cards flipping or revealing.
- Winnings or payouts being visually emphasized.
- Smooth transitions between screens.
- A winning hand briefly scaling or highlighting.

You may use JavaFX animation classes such as:

- `FadeTransition`
- `TranslateTransition`
- `RotateTransition`
- `ScaleTransition`
- `SequentialTransition`
- `ParallelTransition`

Each animation must have a clear UI/UX purpose.

The interface must remain understandable when animations are skipped or reduced.

13. CSS and Visual Design

Use CSS to style the application.

Your visual design should establish consistency across screens in:

- Typography.
- Colors.
- Buttons and controls.

- Spacing.
- Card presentation.
- Status messages.
- Feedback.

You must also implement **a second, noticeably different visual theme**.

Provide a user-facing control to switch themes.

Use CSS stylesheets or style classes rather than individually hard-coding styling throughout the controllers.

14. Automated Testing

Use JUnit Jupiter to test the game logic.

At minimum, your project must include:

- **20 meaningful tests** for `ThreeCardLogic`.
- **10 meaningful tests** for `Deck` and `Dealer` combined.

Tests should cover:

- Hand classifications.
- Hand comparisons and tie-breaking.
- Pair Plus payouts.
- Deck contents and initialization.
- Card dealing.
- Important boundary conditions and edge cases.

Additional automated tests for game-state transitions and wager validation are encouraged.

Tests must verify expected outcomes using assertions.

UML Class Diagram — Required Part 2 Deliverable

Create a UML class diagram representing the **final implementation** of your project.

The diagram must include:

- Major classes and interfaces.
- Important attributes.

- Important public methods.
- Inheritance relationships, where applicable.
- Interface implementations, where applicable.
- Associations and dependencies between major components.

Your diagram should show how the following parts of your application relate to one another:

- Game model.
- Game logic.
- Game coordination/service layer.
- JavaFX controllers.
- Navigation components, if applicable.

The diagram must reflect your actual implementation, not merely your original design intentions.

Submit the UML class diagram as a PDF included in your Part 2 ZIP file.

README and Design Reflection

Include a `README.md` file explaining:

- How to build and run the application.
- How to run the automated tests.
- The responsibilities of the major software components.
- The purpose of each required screen.
- The purpose of each animation.
- Changes made as a result of Part 1 usability testing.

Answer the following design questions:

1. Why did you separate game logic from the FXML controllers?
2. What usability problems did other students identify?
3. Which wireframe revisions most improved the user experience?
4. How do the animations help users understand game events?
5. How could the interface be redesigned without changing the game rules?
6. How does your architecture make the application easier to test and maintain?

For teams of two, identify each member's contributions.

Part 2 Submission

Submit **one ZIP file** containing your completed project to the Part 2 assignment in Canvas by **Sunday, November 15, 2026, at 11:59 PM**. Can submit 24 hours late for a 10% penalty.

Your ZIP file must contain:

1. Complete Maven project.
2. All Java source files.
3. All FXML files.
4. CSS stylesheets for both themes.
5. Images and other required resources.
6. JUnit test files.
7. `pom.xml`.
8. `README.md` with the required design reflection.
9. **UML class diagram PDF** reflecting the final implementation.

Your application must be buildable and runnable from the submitted project.

18. Project Restrictions

- Required interface screens must be defined using FXML.
- Do not place game-rule or payout calculations in controller classes.
- Do not implement the entire application in one class.
- Do not depend on IDE-specific settings to run the application.
- Do not use animations merely as decoration without a meaningful UI/UX purpose.
- Do not omit the required peer usability testing in Part 1.

19. Evaluation Criteria

Your project will be evaluated on:

- Correctness of Three Card Poker gameplay.
- Quality of software architecture and separation of concerns.
- Correct use of FXML, controllers, and JavaFX events.
- Quality of screen design, navigation, and user feedback.
- Application of UI/UX principles.
- Quality of Part 1 usability testing and subsequent revisions.
- Effective use of JavaFX animations.
- CSS consistency and alternate theme.
- Automated unit testing.
- Accuracy and completeness of the final UML class diagram.
- Maven configuration and project completeness.
